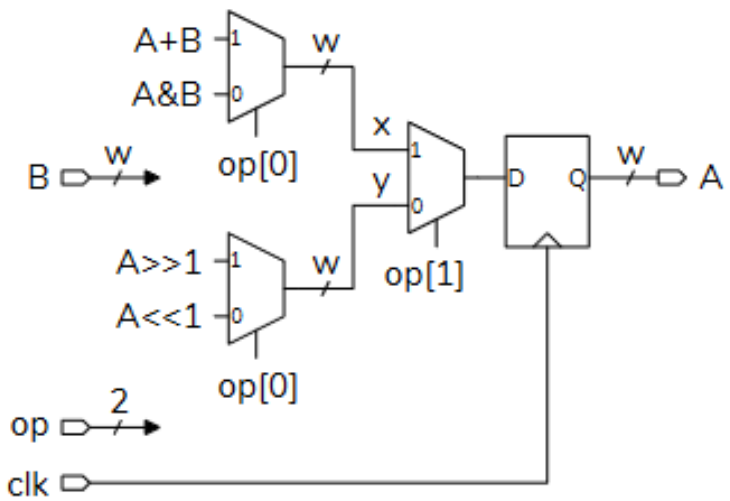
Design a combinational logic circuit which receives a **4-bit** unsigned number X **(x_3 x_2 x_1 x_0)** as input and produces an output Z which equals the result of integer division of X **by 3** (e.g., if X=7, Z=2).

(i) How many bits does the output Z have? Why?

(ii) Derive the truth table of this circuit.

(iii) Using K-maps, derive minimized sum-of-products expression(s) for the circuit output(s).

Write a Verilog module named **alu** corresponding to the diagram shown below. The module has a w-bit input **B**, a 2-bit input **op**, a clock input **clk**, and a w-bit registered output **A**. The module declaration should include a parameter named w for the bus widths with a default value of 16. The use of the signals **x** and **y** is optional but they must be declared if they're used. Bits should be numbered in decreasing order. Follow the course coding conventions. You may omit comments.



Answer

An answer using **x** and **y** is:

```
module alu
  #(parameter w=16)
  ( input logic [w-1:0] B,
    input logic [1:0] op,
    input logic clk,
    output logic [w-1:0] A ) ;

   logic [w-1:0] x, y ;

   assign x = op[0] ? A+B : A&B ;
   assign y = op[0] ? A>>1 : A<<1 ;

   always @(posedge clk) A
     <= op[1] ? x : y ;

endmodule
```

alu

```
 3 module alu #(parameter W = 32)(
 4     input       [1:0] opcode,
 5     input       [W-1:0] operand_A,
 6     input       [W-1:0] operand_B,
 7     output reg  [W-1:0] alu_result,
 8     output      [2:0] flags
 9     );
10
11     wire N, Z, C; // negative zero carry
12     wire [W:0] carry_check;
13
14     always @* begin
15         case(opcode)
16             2'b00 : alu_result = operand_A + operand_B; // add
17             2'b01 : alu_result = operand_A - operand_B; // sub
18             2'b10 : alu_result = operand_A & operand_B; // AND
19             2'b11 : alu_result = operand_A | operand_B; // OR
20         endcase
21     end
22
23     assign carry_check = operand_A + operand_B;
24     assign N = alu_result[W-1];
25     assign Z = ~|alu_result;
26     assign C = carry_check[W];
27     assign flags = {N, Z, C};
28
29 endmodule
```

```
initial begin
    operand_A = 8'd32;
    operand_B = 8'd32;
    opcode = 2'b00;      // ADD
    #2
    opcode = 2'b01;      // SUB, check for Z flag
    #2;

    operand_A = 8'b1010_1010;
    operand_B = 8'b0101_0101;
    opcode = 2'b10;      // AND, check for Z flag
    #2
    opcode = 2'b11;      // OR, check for N flag
    #2;

    operand_A = 8'b1111_1111;
    operand_B = 8'b0000_0001;
    opcode = 2'b00;      // ADD, check for C flag
    #2;

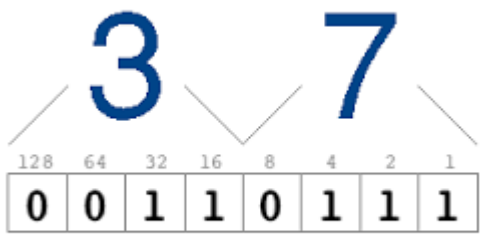
    operand_A = 8'd0;
    operand_B = 8'd0;
    opcode = 2'b00;      // ADD, check for Z flag
    #2;

    operand_A = 8'b0111_1111;
    operand_B = 8'b1111_1111;
    opcode = 2'b01;      // SUB, check for N flag

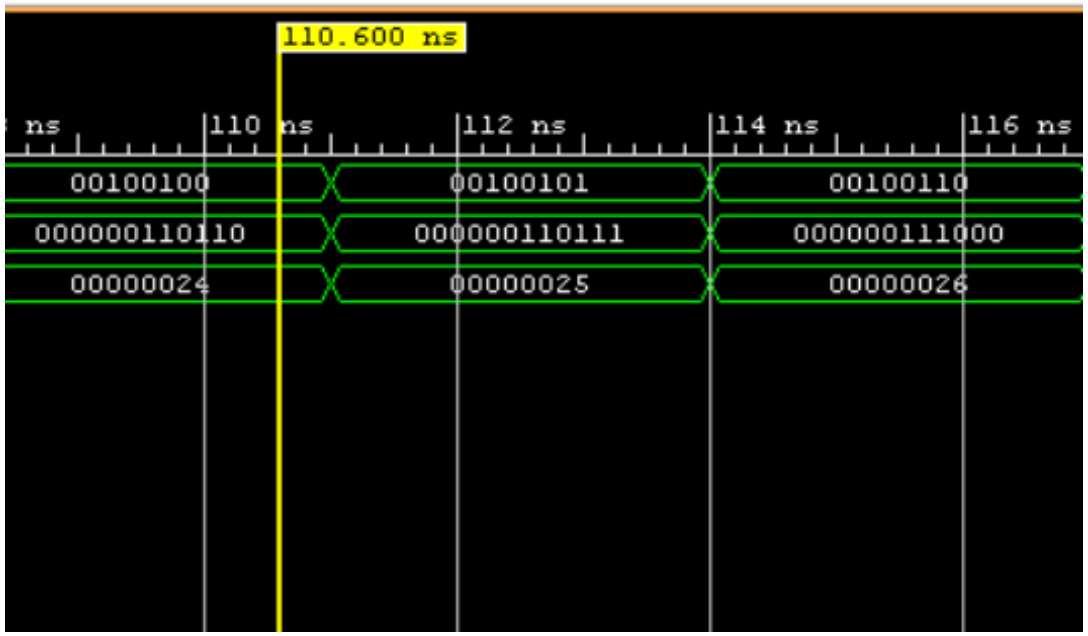
  //  #2 $finish;
end
```

bcd

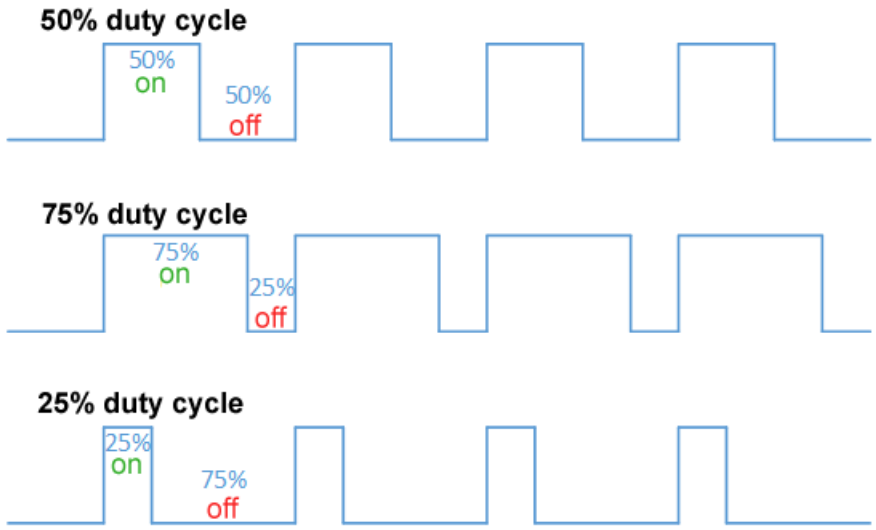
Decimal Number	Binary Number	Binary-Coded Decimal (BCD)
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101



32+16+4+2+1=55



pwm




```
module pwm_gen(
    input clk,
    output pwm30,
    output pwm50,
    output pwm70,
    output pwm90
    );

reg [6:0] counter = 0;      // to hold the value of 99

always @(posedge clk) begin
    if (counter < 100)
    counter <= counter + 1;
    else
    counter <= 0;
end

assign pwm30 = (counter < 30) ? 1 : 0;  // Create a 30% duty cycle
assign pwm50 = (counter < 50) ? 1 : 0;  // Create a 50% duty cycle
assign pwm70 = (counter < 70) ? 1 : 0;  // Create a 70% duty cycle
assign pwm90 = (counter < 90) ? 1 : 0;  // Create a 90% duty cycle

endmodule
```

edge detector

```
module edge_detector(in,rst,clk,out);

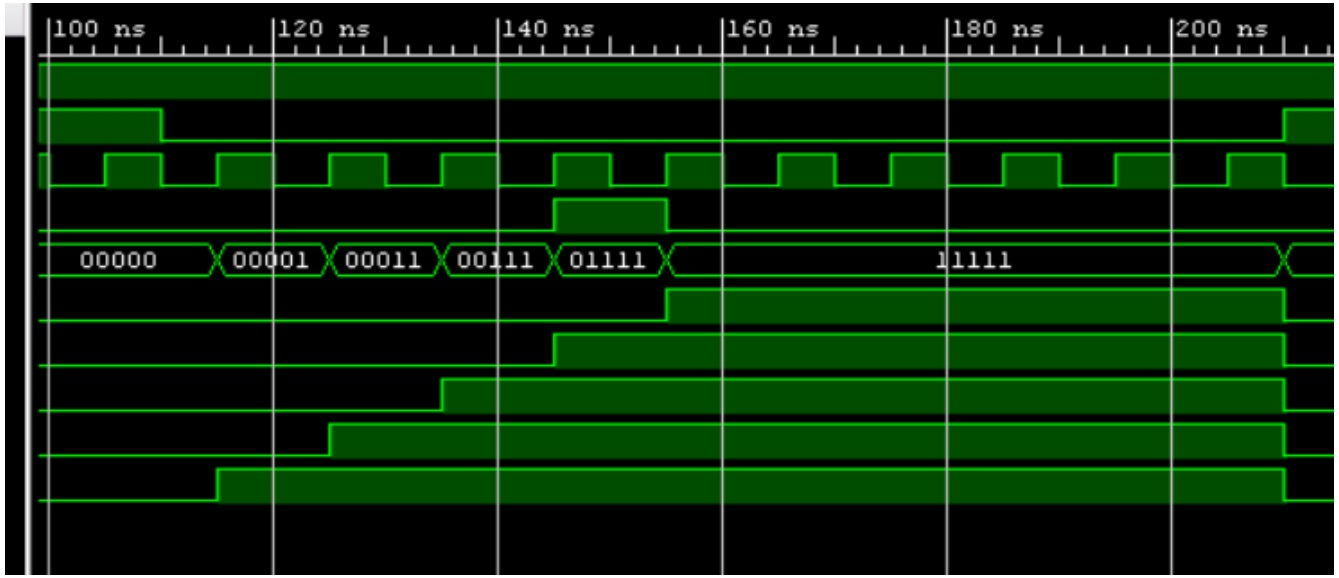
input in,rst,clk;
output out;

reg [4:0]shift_reg;

always @ (posedge clk or posedge rst)
begin
  if(rst == 1) shift_reg <= 0;
  else shift_reg[4:0] <= {shift_reg[3:0],in};
end

assign out = ~shift_reg[4] & shift_reg[3];

endmodule
```



gray counter

Binary Code b[3:0]	Gray Code g[3:0]
0 0 0 0	0 0 0 0
0 0 0 1	0 0 0 1
0 0 1 0	0 0 1 1
0 0 1 1	0 0 1 0
0 1 0 0	0 1 1 0
0 1 0 1	0 1 1 1
0 1 1 0	0 1 0 1
0 1 1 1	0 1 0 0
1 0 0 0	1 1 0 0
1 0 0 1	1 1 0 1
1 0 1 0	1 1 1 1
1 0 1 1	1 1 1 0
1 1 0 0	1 0 1 0
1 1 0 1	1 0 1 1
1 1 1 0	1 0 0 1
1 1 1 1	1 0 0 0

```
module gray_counter(
    input clock,
    input reset,
    output reg [2:0] gray_out
);

    reg [2:0] bin_counter;

    always @(posedge clock or negedge reset) begin
        if(~reset)
            bin_counter <= 0;
        else
            bin_counter <= bin_counter + 1;
    end

    always @(bin_counter) begin
        case(bin_counter)
            3'b000 : gray_out = 3'b000;
            3'b001 : gray_out = 3'b001;
            3'b010 : gray_out = 3'b011;
            3'b011 : gray_out = 3'b010;
            3'b100 : gray_out = 3'b110;
            3'b101 : gray_out = 3'b111;
            3'b110 : gray_out = 3'b101;
            3'b111 : gray_out = 3'b100;
        endcase
    end

endmodule
```

```
assign gray_out[2] = bin_counter[2];
assign gray_out[1] = bin_counter[2] ^ bin_counter[1];
assign gray_out[0] = bin_counter[1] ^ bin_counter[0];
```

fibonacci

The Fibonacci Sequence

1,1,2,3,5,8,13,21,34,55,89,144,233,377…

1+1=2	13+21=34
1+2=3	21+34=55
2+3=5	34+55=89
3+5=8	55+89=144
5+8=13	89+144=233
8+13=21	144+233=377

```
module fibonacci(
    input clock,
    input reset,
    output [15:0] fib_out
);

    reg [15:0] num0, num1;

    always @(posedge clock or negedge reset) begin
        if(~reset) begin
            num0 <= 0;
            num1 <= 1;
            end
        else begin
            num0 <= num1;
            num1 <= fib_out;
            end
    end

    assign fib_out = num0 + num1;

endmodule
```